

Vector Stores In LangChain

Search by meaning,
not by words



CONTENT

Vector Store in LangChain	3
1. Practical Implementation of Vector Store (Chroma)	4
1.1 Setup and ingestion	4
1.2 Initializing Embedding Model	4
1.3 Creating a Vector Store (Chroma)	5
1.4 Document Operations	6
2. Project Implementation (Movie Recommendation System)	9
2.1 Project Overview	9
2.2 Loading Dataset	9
2.3 Initializing Embedding Model	10
2.4 Creating a Vector Store	10
2.5 Indexing/Adding Documents	10
2.6 Similarity Search	10
2.7 Recommendation Function	11
2.8 Frontend with Gradio	11

Vector Store in LangChain

A **vector store** is a specialized system designed to store and retrieve data as mathematical representations called **vector embeddings**. It is a technology that allows AI and search systems to understand meaning, context, and similarity, rather than just matching the exact keywords.

Traditional databases search for exact matches (e.g., SQL queries). A vector store works differently:

1. **Embedding:** Machine learning models (like OpenAI or Hugging Face) convert unstructured data—text, images, or audio—into a sequence of hundreds or thousands of numbers (a vector).
2. **Clustering:** In this high-dimensional mathematical space, items with similar meanings are grouped close together. For example, the vector for “cat” will be mathematically closer to “kitty” than “dog”.
3. **Similarity Search:** When we query the store, it compares our query vector to the stored vectors and retrieves the items that are conceptually closest, rather than spelling matches.

The common use case of vector stores are:

- **Retrieval-Augmented Generation (RAG):** Retrieve relevant documents or context to improve LLM responses.
- **Semantic Search:** Search based on meaning rather than exact keyword matching.
- **Recommendation Systems / Engines:** Recommend similar products, movies, articles, or content based on embeddings.

Some common vector stores:

- **Chroma**
- Pinecone
- Faiss
- Weaviate
- Qdrant and so on.

1. Practical Implementation of Vector Store (Chroma)

Let's learn the basics of Vector Store. We will use **LangChain** to manage the logic and **Chroma** as our database.

1.1 Setup and ingestion

First, we need to import our tools and **create a few example documents** about *Harry Potter*. These documents will eventually be transformed into vectors and stored in our vector store.

```
from langchain_classic.schema import Document

documents = [
    Document(
        page_content="Harry Potter is a young wizard known for surviving an attack by Lord Voldemort as a baby and later attending Hogwarts School of Witchcraft and Wizardry.",
        metadata={"source": "character-bio-doc"},
    ),
    Document(
        page_content="Hermione Granger is one of Harry Potter's closest friends, recognized for her intelligence, bravery, and exceptional magical abilities.",
        metadata={"source": "character-bio-doc"},
    ),
    Document(
        page_content="Ron Weasley is Harry Potter's loyal best friend and a member of the large and well-known Weasley wizarding family.",
        metadata={"source": "character-bio-doc"},
    ),
    Document(
        page_content="Albus Dumbledore is the wise headmaster of Hogwarts and one of the most powerful wizards of his time.",
        metadata={"source": "character-bio-doc"},
    )
]
```

1.2 Initializing Embedding Model

Before storing documents in a vector database, we need to convert the text into numerical representations called **embeddings**. Embeddings capture the semantic meaning of the text, which makes it easier to perform similarity searches and retrieve the relevant information based on meaning rather than exact keyword matches.

There are many embedding models available. Some of popular models are:

- OpenAI Embeddings (text-embedding-3-small, text-embedding-3-large)
- Sentence Transformers (all-MiniLM-L6-v2)
- BGE Embeddings (BAAI/bge-small-en-v1.5, bge-large-en-v1.5)
- Nomic Embeddings (nomic-embed-text) and so on.

For this section, we will use the open-source **all-MiniLM-L6-v2** embedding model from the **Sentence Transformer** library.

```
from langchain_huggingface import HuggingFaceEmbeddings

embedding_model = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
```

Let's quickly generate an embedding:

```
# Generating an embeddings
text = "Hello there!"
embeddings = embedding.embed_query(text)
print("Embedding dimension:", len(embeddings))
print("Embedding:", embeddings[:5])
```

```
Embedding dimension: 384
Embedding: [-0.06473074108362198, 0.0644320473074913,
0.01523186918348074, 0.028408793732523918, -0.06111668050289154]
```

1.3 Creating a Vector Store (Chroma)

```
from langchain_community.vectorstores import Chroma
vector_store = Chroma(
    embedding_function=embedding,
    persist_directory="my_chroma_db", # directory where vectors will be
stored
    collection_name="harry_potter" # Collection name (same as SQL table
name)
)
```

1.4 Document Operations

Once documents have been converted into embeddings and stored in a vector database, we can perform various operations on them. These operations allow us to manage the knowledge stored in the vector store and retrieve relevant information when needed.

1.4.1 Adding documents to vector store

We can add documents to the vector store using the `add_documents()` method. When a document is added, its embedding is automatically generated and stored along with the document content and metadata.

```
vector_store.add_documents(documents)
```

Once the documents are added, this returns the id of each document.

```
[ 'e339f2bb-5aaf-4186-98b9-8594f00c452e',  
  '64d4bbe9-27e4-4bd9-abea-97437ba7b78e',  
  '7347de14-aad3-417d-810c-9e781e176902',  
  'bffc2d9d-fa2d-4064-a371-489f817cb0d3' ]
```

1.4.2 Similarity Search

Now that our documents are stored as vectors, we can perform a **similarity search**. Unlike traditional keyword-based search, similarity search retrieves documents based on their meaning rather than exact word matches.

For example, if a document contains the phrase *"Hermione Granger is highly intelligent"* and the user searches for *"Harry Potter's smartest friend"*, a similarity search can still retrieve the correct document even though the exact words do not match.

```
query = "Harry Potter's smartest friend"  
results = vector_store.similarity_search(  
    query=query,  
    k = 2  
)
```

```
for i, result in enumerate(results, start=1):  
    print(f"Result {i}:")  
    print(result.page_content)  
    print()
```


Result 1:

Hermione Granger is one of Harry Potter's closest friends, recognized for her intelligence, bravery, and exceptional magical abilities.

Result 2:

Albus Dumbledore is the wise headmaster of Hogwarts and one of the most powerful wizards of his time.

1.4.3 Updating Documents

If a document contains outdated or incorrect information, it can be updated. Since the content changes, a new embedding is generated and stored.

```
updated_doc = Document(  
    page_content="Professor Severus Snape is a skilled Hogwarts  
professor known for teaching Potions and his mysterious personality.",  
    metadata={"source": "character-bio-doc"}  
)  
  
vector_store.update_document(  
    document_id="7347de14-aad3-417d-810c-9e781e176902",  
    document=updated_doc)
```

1.4.4 Reading documents

Vector store also allows us to retrieve stored documents directly using the **get()** method. This method returns the stored documents, metadata, IDs, embeddings(if requested).

```
vector_store.get()
```

This will not show embeddings. To get that we should do this:

```
vector_store.get(include=[ "embeddings", "documents", "metadatas",  
"uris"])
```

We can also mention the id to get the specific document

```
vector_store.get(  
    ids="7347de14-aad3-417d-810c-9e781e176902",  
    include=[ "embeddings", "documents", "metadatas", "uris"]  
)
```

1.4.5 Deleting documents

Documents that are no longer needed can be removed from the vector store along with their embeddings.

```
vector_store.delete(ids=[ "7347de14-aad3-417d-810c-9e781e176902" ] )
```


2. Project Implementation (Movie Recommendation System)

2.1 Project Overview

The **Movie Recommendation System** recommends similar movies based on a user-provided movie description or title using **semantic similarity search**.

Instead of traditional recommendation methods (ratings, collaborative filtering, etc.), this system uses:

- **Embeddings** to convert movie descriptions into vectors
- **Vector Database (Chroma)** to store embeddings
- **Similarity Search** to retrieve nearest movies
- **Gradio UI** for user interaction

The system can understand meaning, context, genre, keywords, and themes—not just exact text matches.

Example: When a user inputs “Fantasy movie with witches, magical kingdom, siblings, lion, winter battle”, the output will be “The Chronicles of Narnia”.

2.2 Loading Dataset

We load the JSON file and convert each entry into a LangChain **Document**.

```
import json
from langchain_core.documents import Document

with open("../1_Datasets/movies_data_5000.json", "r", encoding="utf-8")
as f:
    data = json.load(f)

final_docs = [Document(**item) for item in data]
```

2.3 Initializing Embedding Model

```
# We will load the embedding model from huggingface
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name = "Sentence-transformers/all-MiniLM-L6-v2"
)
```

2.4 Creating a Vector Store

```
!uv add chromadb -q
```

```
from langchain_community.vectorstores import Chroma

# Create Chroma vector store
vector_store = Chroma(
    embedding_function=embeddings,
    persist_directory="chroma_db",
    collection_name="movies"
)
```

2.5 Indexing/Adding Documents

```
# Well this part will take time
from tqdm import tqdm # To add progress bar

for i in tqdm(range(0, len(final_docs)), desc="Indexing Movies"):
    vector_store.add_documents([final_docs[i]])
```

2.6 Similarity Search

```
query = """
An 11-year-old boy discovers he is a wizard and begins studying at a
magical boarding school.
As he learns magic and forms lasting friendships, he uncovers the truth
about his family's past
and confronts a powerful dark wizard.
"""

similar_docs = vector_store.similarity_search(
    query=query,
    k=10
)
```

```
# Extracting the titles of the movies
for doc in similar_docs:
    print(doc.metadata["title"])
```

2.7 Recommendation Function

```
TMDB_IMAGE_BASE_URL = "https://image.tmdb.org/t/p/w500"

def recommend_movies(query, num_movies=10):
    similar_docs = vector_store.similarity_search(
        query=query, k=num_movies
    )

    movies =
    [f"{TMDB_IMAGE_BASE_URL}{doc.metadata['poster_path'].strip()}" for doc
in similar_docs]
    return movies
```

2.8 Frontend with Gradio

```
import gradio as gr

with gr.Blocks(theme=gr.themes.Soft()) as demo:

    gr.Markdown("# Movie Recommendation System")

    movie_input = gr.Textbox(
        label="Movie Description",
        placeholder="Enter a movie title or description..."
    )

    search_button = gr.Button("Search similar Movies", variant="primary")

    gallery = gr.Gallery(
        label = "Recommended Movies", columns=5, height=500)

    search_button.click(
        fn = recommend_movies,
        inputs = movie_input,
        outputs = gallery
    )

demo.launch()
```